Real Time Data Acquisition and analysis of Engine performance and vehicle Load using CAN protocol

Submitted
In partial fulfilment

For the award of the Degree of

PG-Diploma in Embedded Systems and Design (PG-DESD)

C-DAC, ACTS (Pune)

Guided By:	**Submitted By:**	
Mr. Rhugved Rane	Anagha Kolhe	[250240130003]
	Ayush Manoj Pudke	[250240130006]
	Prateek Kumar Singh	[250240130022]
	Samadhan Pandharinath Dongare	[250240130031]
	Sourabh Sanjay Patil	[250240130037]

Centre for Development of Advanced Computing (C-DAC), ACTS

(Pune- 411008)

ACKNOWLEDGEMENT

I would like to express my heartfelt gratitude to my project guide, Rhugved Rane, for their invaluable support, timely suggestions, and encouragement throughout the course of this project. Their insightful feedback greatly enhanced the quality of this work. I am also grateful to the faculty members of the Department of Embedded Systems and Design for providing me with the resources and guidance needed to complete this project.We extend my sincere thanks to my teammates for their dedication and teamwork, without which the project would not have been possible.

ABSTRACT

This project presents the design and implementation of a real-time system for the acquisition and analysis of engine performance and vehicle load using the Controller Area Network (CAN) protocol. The system is composed of multiple ESP32-WROOM microcontroller nodes interfaced with a set of vehicle-mounted sensors, including temperature sensors, float sensors, load cells, and door sensors. These distributed nodes communicate over the MCP2515-based CAN bus, ensuring reliable and efficient data transfer within the vehicle's embedded sensor network.

Sensor data from each node is transmitted to a central ESP32-WROOM controller, where key parameters are displayed locally on a TFT display. To enable remote monitoring and data logging, the central node uses Wi-Fi or GSM (SIM800L) connectivity to upload data to the ThinkSpeak IoT cloud platform via REST API calls. By encoding sensor readings in JSON format and posting them regularly to ThinkSpeak, the system supports real-time dashboards, field-wise visualization, and historical analytics.

The system integrates critical automotive parameters such as engine temperature, fuel level, vehicle load, and door status, making it suitable for use in vehicle diagnostics, logistics, and preventive maintenance applications. The modular design allows easy expansion to include additional sensor types or nodes in the future. With reliable cloud integration, fast CAN-based communication, and redundant communication links, this project establishes a practical foundation for intelligent and connected vehicular monitoring systems.

Table of Contents

Chapter 1: Introduction

The rapid advancement in automotive electronics has created a pressing need for efficient data acquisition systems capable of monitoring and analyzing various vehicle parameters in real-time. Modern vehicles are becoming increasingly complex with the integration of multiple electronic control units (ECUs), sensors, and actuators that ensure optimal performance, safety, and efficiency. Key operational parameters such as engine temperature, fuel levels, and load conditions must be continuously monitored to ensure the smooth functioning of vehicles and to enable early fault detection.

One of the most reliable and widely adopted communication protocols for such in-vehicle networking is the Controller Area Network (CAN) protocol. Developed by Bosch in the 1980s, CAN is a robust serial communication protocol that enables microcontrollers and devices to communicate with each other without the need for a host computer. It supports real-time data exchange and prioritization, making it suitable for critical automotive applications. CAN is particularly favored for its error detection capabilities, deterministic message delivery, and ability to handle multiple nodes on a single bus.

In this project, we leverage the capabilities of the ESP32-WROOM microcontroller, a low-power system-on-chip with integrated Wi-Fi and Bluetooth, to build a distributed vehicle monitoring system. Each ESP32 node is equipped with specific sensors and communicates over the CAN network using MCP2515 transceivers, which interface the ESP32 with the CAN protocol via SPI. The system is designed to collect data from temperature sensors, float sensors, load cells, and door sensors, reflecting various aspects of engine performance and vehicle conditions.

The central ESP32-WROOM node acts as the master controller, receiving and interpreting CAN messages from all other sensor nodes. This node is equipped with a TFT display for real-time data visualization and a SIM800L GSM module that ensures continuous cloud connectivity via the ThinkSpeak platform. This setup provides real-time analytics and logging capabilities accessible from remote locations, which is particularly beneficial for fleet operators, logistics companies, and vehicle diagnostics services.

By combining sensor technology, wireless communication, and cloud computing, this system enhances the overall transparency, safety, and intelligence of modern transportation systems. It also opens pathways for predictive maintenance, automatic fault reporting, and smart vehicle analytics, contributing to the development of

next-generation vehicular monitoring infrastructures.

1.1 Objective and Specifications

The primary objective of this project is to design and implement a real-time vehicle monitoring system capable of acquiring, transmitting, and analyzing critical engine and vehicle parameters. The ultimate goal is to create an integrated solution that not only tracks vital performance metrics but also facilitates intelligent decision-making through cloud-based data analysis. The project aims to enable both local and remote access to real-time sensor data for enhanced vehicle management, maintenance, and diagnostics.

Specific objectives of the project include:

- Developing a distributed sensor network using ESP32-WROOM microcontrollers, with each unit dedicated to monitoring a particular subsystem of the vehicle.

- Implementing robust and fault-tolerant CAN-based communication between nodes using MCP2515 CAN transceivers.

- Acquiring real-time data from sensors including a temperature sensor (engine heat monitoring), float sensor (fuel level monitoring), load cell (vehicle payload tracking), and a magnetic door sensor (security status reporting).

- Displaying real-time sensor information graphically on a TFT screen mounted in the vehicle.

- Uploading collected data to the ThinkSpeak cloud platform using HTTP/MQTT REST APIs over Wi-Fi or GSM (SIM800L) to provide continuous remote connectivity.

Chapter 2: Literature Review

Numerous studies have emphasized the importance of real-time data acquisition in modern vehicles. Traditional methods, although effective to some extent, lack the flexibility, scalability, and remote access capabilities offered by microcontroller-based distributed systems. These older systems often relied on wired connections to central ECUs with limited data logging and lacked cloud connectivity.

The introduction of the CAN protocol in automotive applications since the 1980s marked a significant technological leap. CAN's robust communication features, such as multi-master architecture, message prioritization, and error checking, have made it the standard in automotive networking. It has been widely adopted in vehicles for real-time sensor data exchange between various electronic control units (ECUs).

Recent research emphasizes the role of IoT in transforming conventional automotive monitoring into intelligent systems. ESP32 microcontrollers have become a preferred choice for such IoT implementations due to their dual-core processing capabilities, built-in wireless communication, low power consumption, and extensive open-source support. These features make them ideal for developing edge computing applications in vehicles.In parallel, cloud computing technologies such as ThingSpeak IoT Core provide a robust platform for managing IoT devices, processing sensor data in real-time, and enabling remote access and control. The integration of MQTT protocol allows for efficient data transmission with minimal bandwidth consumption, making it suitable for automotive use cases where data packets need to be lightweight and timely.

Several academic projects and industry prototypes have explored CAN-based systems with cloud connectivity. However, most implementations focus on diagnostics rather than real-time telemetry. This project fills that gap by combining sensor fusion, distributed CAN communication, edge processing via ESP32, and cloud analytics—thus creating a comprehensive end-to-end vehicle data acquisition platform.

Studies have also shown that such systems can significantly reduce vehicle downtime by enabling predictive maintenance. By monitoring key parameters and analyzing trends in cloud-based systems, potential failures can be detected early, saving cost and improving road safety.

Chapter 3: Methodology and Techniques

This chapter outlines the systematic approach adopted to design, develop, and implement the real-time data acquisition and analysis system. The methodology integrates embedded systems, communication protocols, and cloud services into a unified framework. It involves a modular design where each CAN node is dedicated to specific tasks and sensors, and collectively they ensure comprehensive data collection and monitoring.

3.1 Design Strategy

The system is architected with a modular and scalable design that divides the sensing responsibilities among three dedicated nodes. These nodes work collaboratively to gather, format, and transmit data related to vehicle operation, environmental conditions, and geolocation. Each node is responsible for a subset of parameters and is constructed with minimal hardware complexity to ensure low cost and high reliability in real-world deployment.

- **Node 1:** This node focuses on engine performance monitoring. It is connected to a temperature sensor to measure engine heat and a float sensor to monitor fuel level inside the tank. These values are crucial for engine diagnostics and vehicle range estimation.
- **Node 2:** Designed to monitor the structural and operational load on the vehicle. It also incorporates a magnetic reed switch to detect the open/closed status of the vehicle door, which enhances security and safety monitoring.

Each of these nodes is built around an ESP32-WROOM microcontroller and connected to an MCP2515 CAN controller. The CAN messages from each node are uniquely tagged with identifiers to help the central node interpret and organize incoming data effectively. The ESP32 gathers sensor data and transmits it in the form of CAN messages.

3.2 Communication Architecture

The CAN network is implemented using MCP2515 modules that communicate over the SPI bus with the ESP32 microcontrollers. The use of CAN ensures deterministic, reliable, and multi-node communication, essential for real-time applications. Each message is framed with a unique identifier (ID) to distinguish sensor types.

3.3 Central Node and Data Processing

The central ESP32-WROOM node receives messages from the CAN bus, parses the sensor data, and displays it on a TFT screen for local monitoring. The central node is also responsible for data aggregation and formatting for cloud upload.

3.4 Cloud Connectivity and IoT Integration

The central node sends real-time data to the ThinkSpeak cloud platform using HTTP or MQTT-based REST APIs. Data can be uploaded through Wi-Fi or via GSM (SIM800L) to ensure continuous cloud communication even in areas without Wi-Fi coverage. Sensor values are formatted in JSON and mapped to ThinkSpeak fields for online storage, graph plotting, and dashboard visualization.

3.5 Hardware Interfacing

The interfacing of each sensor to the ESP32 is carefully designed:

- Float sensors use digital input.
- Load cell data is converted using the HX711 module.
- Magnetic door sensor uses GPIO.
- GPS module communicates using UART protocol.

Power is supplied through a regulated 12V–5V–3.3V rail design, ensuring stable operation for logic and analog components.

3.6 Software and Programming

The codebase is written in Arduino IDE using C/C++. Libraries used include:

- "OneWire" and "DallasTemperature" for DS18B20
- "Adafruit_GFX" and "TFT_eSPI" for display
- "TinyGPS++" for GPS parsing
- "PubSubClient" and "WiFiClientSecure" for MQTT communication

The firmware also incorporates interrupt-based data acquisition and CAN bus timing optimization to improve responsiveness and reduce latency.

3.7 Component Overview:

3.7.1 ESP32-WROOM-32:

The ESP32-WROOM-32 is a powerful, generic Wi-Fi + Bluetooth + Bluetooth LE MCU module manufactured by Espressif Systems. At its core is the ESP32-D0WDQ6 chip, a highly integrated System-on-a-Chip (SoC) designed for a wide variety of applications, from low-power sensor networks to more demanding tasks like voice encoding and video streaming.

The module is a self-contained solution, featuring an integrated PCB antenna, a 4 MB SPI flash memory, and a 40 MHz crystal oscillator. This makes it an ideal foundation for Internet of Things (IoT), home automation, and other embedded systems projects.

Figure 1 : ESP32 Wroom-32

Key Features and Specifications

- **Processor:** Dual-core Xtensa® 32-bit LX6 microprocessor with a clock frequency adjustable from 80 MHz to 240 MHz. One core is typically used for high-speed connectivity, while the other handles application logic.
- **Wireless Connectivity:**
    - **Wi-Fi:** 802.11 b/g/n, supporting a data rate of up to 150 Mbps.
    - **Bluetooth:** Dual-mode support for both classic Bluetooth v4.2 BR/EDR and Bluetooth Low Energy (BLE).

- **Memory:**
    - 520 KB of internal SRAM.
    - 448 KB of ROM for booting and core functions.
    - 4 MB of integrated SPI flash memory for program storage.
- **Power Consumption:** The module features multiple power-saving modes, with a deep-sleep current as low as 5 µA, making it highly suitable for battery-powered applications.
- **Peripheral Interfaces:** The ESP32 offers a rich set of peripherals, including:
    - **GPIOs:** Up to 34 programmable General-Purpose Input/Output pins.
    - **ADC:** Two 12-bit Analog-to-Digital Converters with up to 18 channels.
    - **DAC:** Two 8-bit Digital-to-Analog Converters.
    - **Touch Sensors:** 10 capacitive touch-sensing GPIOs.
    - **Communication Protocols:** Multiple interfaces for SPI, I2C, I2S, UART, SD card, and PWM.
- **Security:** Features include secure boot, flash encryption, and cryptographic hardware acceleration (AES, SHA-2, RSA, ECC).

3.7.2 MCP2515

The MCP2515 is a standalone Controller Area Network (CAN) controller produced by Microchip Technology. It is designed to simplify the implementation of a CAN communication interface in embedded systems that do not have a built-in CAN controller. The MCP2515 handles the complexities of the CAN protocol, allowing a host microcontroller (MCU) to send and receive CAN messages via a simple and widely-supported Serial Peripheral Interface (SPI).

To function as a complete CAN node, the MCP2515 must be paired with an external CAN transceiver (such as the TJA1050 or MCP2551). The controller handles the data link layer of the CAN protocol, while the transceiver handles the physical layer, converting the digital signals from the MCP2515 into the differential voltage levels required for the physical CAN bus.

Figure 2 : MCP2515

Key Features and Specifications

- **Protocol Support:** Implements the CAN specification, version 2.0B, and is capable of transmitting and receiving both standard (11-bit) and extended (29-bit) frames.
- **SPI Interface:** Communicates with a host microcontroller using a high-speed SPI interface, supporting clock speeds up to 10 MHz.
- **Receive Buffers:** Features two receive buffers with a prioritized message storage system. This allows the MCP2515 to store incoming messages while the host MCU is busy.
- **Filters and Masks:** Includes two acceptance masks and six acceptance filters. These are used to filter out unwanted messages based on their identifiers, reducing the processing overhead on the host microcontroller.
- **Transmit Buffers:** Has three transmit buffers with prioritization capabilities. This allows the host MCU to queue multiple messages for transmission, and the MCP2515 will handle sending them based on their assigned priority.

- **Interrupt Functionality:** An interrupt output pin (INT) can be configured to alert the host MCU of important events, such as a message being received, a message being transmitted, or an error occurring. This allows for a more efficient, interrupt-driven system rather than continuous polling.
- **Operating Voltage:** Operates within a wide voltage range of 2.7V to 5.5V, making it compatible with both 3.3V and 5V microcontrollers.
- **Data Rate:** Supports CAN communication speeds up to 1 Mb/s.
- **Low Power:** Features a low-power standby mode with a typical current draw of only 1 µA.

Functional Description

The MCP2515 acts as an intermediary between a microcontroller and the CAN bus. Its primary functions can be broken down into three main blocks:

1. **SPI Interface Block:** This is the communication link to the host microcontroller. The MCU sends commands to the MCP2515 via the SPI bus to configure the device (e.g., setting the baud rate, filters, and masks), write messages to be transmitted into the transmit buffers, and read received messages from the receive buffers.
2. **CAN Protocol Engine Block:** This is the core of the chip. It manages the complexities of the CAN protocol, including:
    - **Message Handling:** Formatting messages for transmission and de-framing them upon reception.
    - **Arbitration:** Managing the bus access arbitration process, where messages with lower IDs have higher priority.
    - **Error Detection and Handling:** The engine includes hardware for error checking (CRC) and automatically re-transmits messages that encounter errors on the bus.
3. **Transceiver Interface Block:** This block provides the low-level digital interface to the CAN transceiver. The `TXCAN` and `RXCAN` pins are used to exchange data with the transceiver, which then drives the physical CAN-High (`CANH`) and CAN-Low (`CANL`) wires of the bus.

3.7.3 TFT Display Module

The 1.5-inch TFT (Thin-Film Transistor) display is a small, full-color graphic display module designed for use with microcontrollers and embedded systems. It provides a vibrant visual output for user interfaces, data visualization, and simple graphical applications. This particular model typically features a resolution of 128x128 pixels and operates with a built-in display controller, such as the ST7735S. The module communicates with the host microcontroller via a high-speed Serial Peripheral Interface (SPI), which minimizes the number of pins required for data transfer and simplifies wiring.

The TFT technology offers better contrast, brightness, and response time compared to older LCD technologies, making it suitable for applications that require dynamic and clear visual feedback.

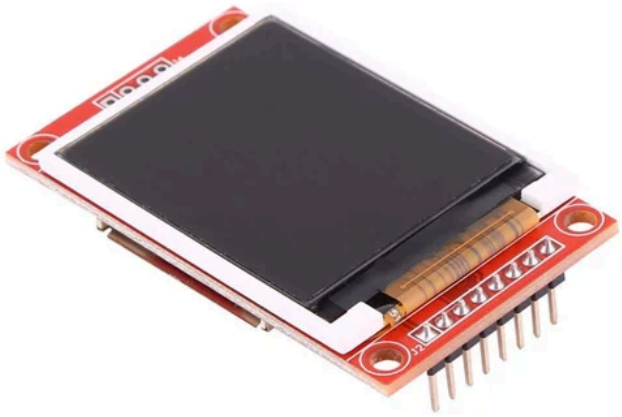

Figure 3: TFT Display Module

Key Features and Specifications

- **Display Size:** 1.5-inch diagonal.
- **Resolution:** 128x128 pixels.
- **Color Depth:** 16-bit (RGB565) or 18-bit color, capable of displaying up to 65,536 colors.

- **Driver IC:** ST7735S (or a compatible driver). This chip handles all display-specific functions, including pixel addressing, color data management, and gamma correction.
- **Interface:** 4-wire SPI (Serial Peripheral Interface). This is the standard communication protocol for this module, enabling fast data transfer with minimal wiring.
- **Operating Voltage:** Typically 3.3V. The logic levels for the control signals are also 3.3V, making it directly compatible with popular microcontrollers like the ESP32 and Arduino Due without the need for level shifters.
- **Backlight:** Integrated LED backlight for illumination, typically controlled by a dedicated pin.
- **Form Factor:** The display comes as a complete module with a PCB that includes the driver chip, supporting components, and a standard pin header for easy breadboard or PCB mounting.

Functional Description

The 1.5-inch TFT display module operates by receiving commands and data from a host microcontroller via the SPI bus. The communication process is managed by the following key pins:

- **SPI Bus (SCL, SDA):** The microcontroller sends a stream of bytes to the display's driver chip. SCL provides the clock signal, and SDA carries the data.
- **Chip Select (CS):** This pin is held low to enable communication with the display. It allows multiple SPI devices to be connected to the same bus, with the microcontroller selecting the desired device by controlling its CS pin.
- **Data/Command (DC):** This is a crucial pin that determines the nature of the data being transmitted on the SPI bus. The microcontroller must set this pin to a low state to send a command (e.g., to set a display window or a color mode) and to a high state to send pixel color data.
- **Reset (RES):** This pin is used to initialize the driver chip to a known state, typically during the power-on sequence.

To display an image or text, the microcontroller sends a series of commands to configure the display followed by a continuous stream of color data for each pixel within that window. The driver chip, ST7735S, then takes this data and drives the individual transistors of the TFT array to display the corresponding colors. The backlight, controlled by the BLK pin, provides the necessary illumination for the display to be visible.

3.7.4 MAX6575 Temperature Sensor

The MAX6575 is a specialized, low-cost, and low-power temperature sensor from Analog Devices (formerly Maxim Integrated) that converts temperature into a proportional time delay. It features a unique single-wire digital interface, allowing a microcontroller to read temperature data from multiple sensors using only a single I/O line. This simplifies wiring and is particularly useful in systems with a limited number of available I/O pins, such as portable or distributed sensor networks. The sensor is designed for applications requiring temperature monitoring of microprocessors, battery packs, and other electronic components.

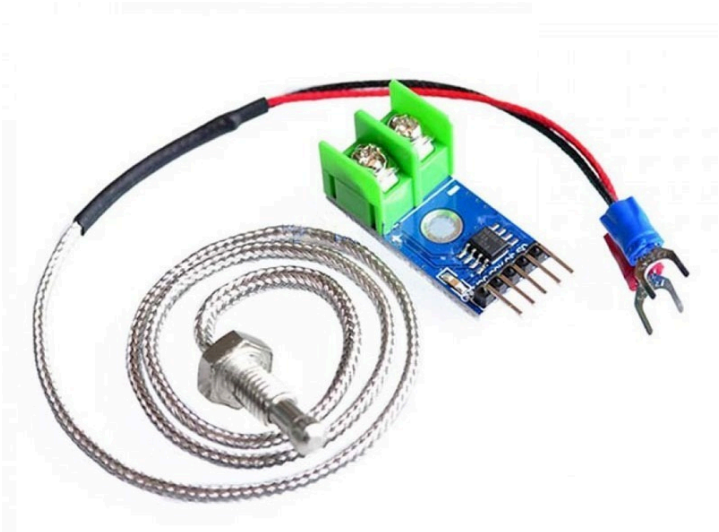

Figure 4: MAX6575 Temperature Sensor

Key Features and Specifications

- **Operating Principle:** The sensor converts temperature into a measurable time delay. The delay is proportional to the absolute temperature in Kelvin (TK).
- **Single-Wire Interface:** Up to eight MAX6575 sensors can be connected to a single microcontroller I/O pin. This is achieved by using different "timeout multipliers" for each device to prevent signal overlap.
- **Temperature Range:** The device is capable of measuring temperatures from -40°C to +125°C.
- **Supply Voltage:** Operates from a single supply voltage of +2.7V to +5.5V.

- **Low Power Consumption:** The typical supply current is only 150 µA, making it suitable for battery-powered applications.
- **Package:** The sensor is available in a small, 6-pin SOT23 package, ideal for space-constrained designs.
- **Programmable Delay Factor:** The device's output delay factor can be configured using two time-select pins, allowing for multiple sensors on the same line.

Functional Description

The MAX6575 functions as a triggered monostable timer where the timing is proportional to temperature. The measurement process is as follows:

1. **Initiating a Measurement:** The microcontroller (µC) initiates a temperature reading by pulling the bidirectional **I/O** pin low for a brief period (a "start pulse").
2. **Conversion Delay:** The MAX6575 recognizes the falling edge of this start pulse and begins an internal timing sequence. After a time delay (t_D), which is proportional to the absolute temperature, the MAX6575 pulls its **I/O** pin low again.
3. **Reading the Temperature:** The microcontroller, having sent the start pulse, waits for the subsequent falling edge from the MAX6575. It measures the total time delay (t_D) between its own start pulse and the sensor's response pulse.
4. **Calculation:** The temperature in degrees Celsius (◦C) can be calculated using the following formula:
   T(◦C)=(Timeout MultipliertD)−273.15
   The "Timeout Multiplier" is a value in µs/◦K (microseconds per degree Kelvin) that is set by the state of the TS0 and TS1 pins. Different multipliers are available for the 'L' and 'H' versions of the chip, allowing up to eight sensors to be used on a single wire by assigning each a unique multiplier.

3.7.5 Float Sensor (Mechanical Float Switch)

A float sensor is a device used to detect the level of a liquid in a container, such as a tank or a sump. The most common type, a mechanical float switch, is a simple and reliable component that acts as a switch, changing its state from open to closed (or vice versa) when the liquid level causes a float to rise or fall past a certain point. This type of sensor is widely used for applications like controlling a pump, activating an alarm, or monitoring fluid levels in a wide range of systems, from industrial machinery to home aquariums.

The sensor typically consists of a hermetically sealed reed switch inside a hollow, non-magnetic stem, with a magnetic float that slides along the stem. The float's position is determined by the liquid level.

Figure 5: Float Sensor

Key Features and Specifications

- **Operating Principle:** A permanent magnet is embedded within the float. As the float rises or falls with the liquid, the magnet's proximity triggers the internal reed switch to either open or close, completing or breaking an electrical circuit.
- **Construction:**
    - **Stem:** Typically made of non-corrosive materials like plastic (polypropylene, PVC) or stainless steel, making it suitable for a variety of liquids, including water, oils, and some chemicals.
    - **Float:** A sealed, buoyant component containing the magnet.
- **Switch Type:** The sensor can be configured as either:
    - **Normally Open (NO):** The switch is open (no continuity) when the float is at its rest position (e.g., low liquid level). When the float rises, the switch closes.
    - **Normally Closed (NC):** The switch is closed (continuity) when the float is at its rest position. When the float rises, the switch opens.
- **Electrical Characteristics:**
    - **Switching Voltage:** Typically rated for low-voltage DC applications (e.g., 5V to 24V), but can also be used with AC circuits depending on the model.

- **Switching Current:** The maximum current the switch can safely handle, often in the range of 0.5A to 1A. It is important to use the sensor with a low-current control circuit (like a microcontroller I/O pin) and a relay for high-current loads.
- **Connection:** A simple two-wire interface.

Connection Details

Connecting a mechanical float switch to a microcontroller is straightforward, as it functions like any other switch. The sensor does not require a power supply; it simply completes or breaks a circuit.

A typical connection to a microcontroller (e.g., an Arduino or ESP32) involves a pull-up or pull-down resistor to ensure a stable digital signal.

Example Connection (Normally Open):

- Connect one wire from the float sensor to a digital input pin on the microcontroller.
- Connect the second wire of the sensor to **GND**.
- Enable the microcontroller's internal pull-up resistor on that digital input pin.

When the liquid level is low, the switch is open, and the input pin reads a `HIGH` state (pulled up to VCC). When the liquid level rises, the switch closes, connecting the pin to GND, and the input pin reads a `LOW` state.

Functional Description

The operation of the mechanical float switch is binary and depends on the orientation of the sensor.

- **Low-Level Detection:** When the float switch is mounted to detect a low liquid level, the float rests at the bottom of the stem. In this state, the internal magnet is positioned away from the reed switch. As the liquid level drops, the float moves downwards with it. When it reaches the specified trigger point, the magnet moves close to the reed switch, causing it to change its state. This is useful for triggering a pump to fill a tank.

- **High-Level Detection:** When the float switch is mounted to detect a high liquid level, the float rests at the bottom of the stem when the tank is empty. As the liquid level rises, the float is lifted. When it reaches the specified trigger point, the magnet's position causes the reed switch to change state. This is useful for triggering a pump to drain a tank or to sound an overflow alarm.

3.7.6 16x2 LCD with I2C Interface

A 16x2 LCD (Liquid Crystal Display) is a standard character display module capable of showing 16 characters on two separate lines. Its primary function is to provide a simple visual output for microcontrollers, displaying text, numbers, and custom characters. The traditional parallel interface for this type of display requires a large number of digital I/O pins (typically 6 to 11), which can be a limitation in embedded systems.

To address this, the I2C backpack was developed. This small circuit board, attached to the back of the LCD, integrates an I/O expander chip, most commonly the PCF8574. This backpack converts the LCD's parallel interface into a two-wire I2C serial interface. This drastically reduces the number of pins required to control the display from over ten to just four: two for power and two for data. This simplification is highly beneficial for projects with limited I/O resources.

Figure 6: 16x2 LCD with I2C Interface

Key Features and Specifications

- **Display Type:** Character-based LCD with a parallel interface.
- **Display Matrix:** 16 characters per line, 2 lines total (16x2).
- **Controller:** Built-in HD44780-compatible controller, which is the industry standard for character LCDs.
- **Backlight:** Integrated LED backlight, typically in green/yellow, blue, or white.
- **Interface:** I2C (Inter-Integrated Circuit) serial communication.
- **I/O Expander:** PCF8574 or a compatible I/O expander chip.
- **Power Supply:** Typically operates on a +5V DC power supply.
- **I2C Address:** The I2C address is configurable, often with a default address of `0x27` or `0x3F` depending on the manufacturer. Solder jumpers on the backpack can be used to change the address, allowing multiple I2C devices to share the same bus.
- **Onboard Potentiometer:** Includes a small potentiometer to manually adjust the display's contrast.

Functional Description

The functional operation of the 16x2 LCD with an I2C backpack is based on a serial-to-parallel data conversion:

1. **I2C Communication:** The microcontroller sends a command or data to the PCF8574 I/O expander chip via the I2C bus. Each transmission consists of the backpack's I2C address and a byte of data.
2. **Parallel Conversion:** The PCF8574 receives the 8-bit data from the microcontroller and translates it into a parallel signal on its 8 output pins (P0-P7).
3. **Pin Mapping:** These output pins are pre-wired to the LCD's control pins:
    - **RS (Register Select)**: Differentiates between commands and data.
    - **RW (Read/Write)**: The backpack typically holds this pin low, as the LCD is only written to.
    - **EN (Enable)**: Triggers the LCD to read the data on the bus.
    - **D4-D7:** The 4-bit data bus.
    - **Backlight:** One of the PCF8574 pins controls the backlight transistor.
4. **LCD Control:** The LCD's built-in HD44780 controller receives these parallel signals and updates the display accordingly. This process happens automatically at the hardware level, with the microcontroller only needing to send the correct commands and character data over the I2C bus.

3.7.7 Load Sensor

It consists of two primary components:

1. Strain Gauge: The fundamental transducer that converts physical force or pressure into a tiny electrical signal (change in resistance).
2. HX711 Module: A dedicated 24-bit analog-to-digital converter (ADC) and amplifier designed to read the minute voltage changes from the strain gauge and convert them into a digital value that a microcontroller can process.

These two components work together as a complete measurement system. The strain gauge or load cell acts as the sensing element, and the HX711 acts as the signal conditioning and data acquisition unit.

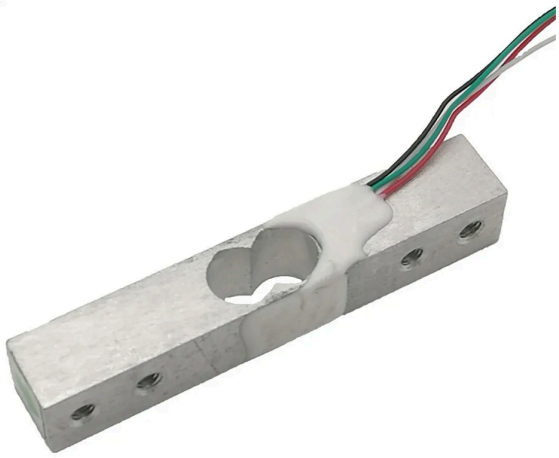

Figure 7: Load Sensor

Strain Gauge

A strain gauge is a passive electrical component whose resistance changes proportionally to the strain (i.e., deformation) applied to it. Strain gauges are typically arranged in a Wheatstone bridge configuration to maximize sensitivity and compensate for temperature changes. A common application uses four strain gauges bonded to a load cell to form a full-bridge circuit.

Key Features

- Operating Principle: Piezoresistive effect, where the electrical resistance changes as the material is stretched or compressed.
- Gauge Factor (GF): A measure of the sensitivity of the strain gauge. It is the ratio of the fractional change in electrical resistance to the fractional change in length. A typical value is around 2.0.
- Nominal Resistance: Standard resistance values are 120 Ω, 350 Ω, and 1000 Ω. The 350 Ω version is a common choice for load cells.
- Configuration: Commonly used in a full-bridge configuration (four strain gauges) to produce a measurable output voltage change when a force is applied.

HX711 Module

The HX711 is a dedicated 24-bit precision ADC specifically designed for weight scales and industrial control applications that interface directly with a bridge sensor. It provides all the necessary signal conditioning, including amplification, to read the tiny voltage changes from a strain gauge accurately.

Key Features and Specifications

- **ADC Resolution:** High-resolution 24-bit analog-to-digital conversion, allowing for extremely precise weight readings.
- **Programmable Gain Amplifier (PGA):** Contains a built-in PGA to amplify the low-level signal from the strain gauge.
    - **Channel A:** Can be set to a gain of **128** or **64**. A gain of 128 is a common choice for load cells.
    - **Channel B:** Has a fixed gain of **32**.
- **Input Channels:** Features two selectable differential input channels (Channel A and B).
- **Digital Interface:** A simple, two-wire serial interface, consisting of a Clock (SCK) and Data (DOUT) pin. This minimizes pin usage on the host microcontroller.
- **Power Supply:** Operates on a supply voltage from 2.6V to 5.5V.
- **Connection:** Provides dedicated pins for both the strain gauge's power (E+, E-) and signal (A+, A-).

System Integration and Functional Description

The strain gauge and HX711 work together as follows:

1. The HX711 provides a stable excitation voltage to the strain gauge or load cell through the E+ and E- pins.
2. When a force is applied to the strain gauge, its resistance changes, causing a small, differential voltage change across its output wires.
3. These differential output wires are connected to the A+ and A- inputs of the HX711.
4. The HX711's internal PGA amplifies this tiny differential voltage.
5. The amplified signal is then converted into a high-resolution, 24-bit digital value by the ADC.
6. The microcontroller, using the SCK and DOUT pins, reads this digital value. The HX711's DOUT pin acts as a digital signal that goes HIGH when the data is ready and is clocked out by the microcontroller using the SCK pin.

3.7.8 Door Sensor (Magnetic Reed Switch)

A door sensor is a device used to detect the state of an opening, such as a door, window, or cabinet. The most widely used type is a magnetic reed switch sensor. This sensor is a two-part component: one part containing a hermetically sealed reed switch and the other containing a permanent magnet. The sensor is non-contact, meaning the two parts do not need to physically touch to operate. When the door is closed, the two parts are aligned, and the magnet holds the switch in a certain state. When the door is opened, the parts separate, and the switch changes its state, signaling the change to a connected system.

This type of sensor is simple, reliable, and requires no power to operate as a basic switch, making it an ideal component for home automation, security systems, and other embedded applications.

Figure 8:Door Sensor

Key Features and Specifications

- **Operating Principle:** The sensor uses the Hall effect principle. The magnetic field of the permanent magnet causes the reed switch's flexible ferrous contacts to either attract or repel, thereby closing or opening an electrical circuit.
- **Construction:**
    - **Reed Switch Housing:** A plastic housing containing the sealed glass tube with the reed switch contacts. This part is typically mounted on the stationary frame of the door.
    - **Magnet Housing:** A corresponding plastic housing containing a permanent magnet. This part is mounted on the moving part of the door.
- **Switch Type:** The sensor functions as a simple electrical switch and can be configured as either:
    - **Normally Open (NO):** The switch is open (no continuity) when the magnet is present (door closed). When the magnet is moved away (door opens), the switch closes.
    - **Normally Closed (NC):** The switch is closed (continuity) when the magnet is present. When the magnet is moved away, the switch opens.
    - *Note: Most consumer models are Normally Closed, as it allows a security system to detect a break in the circuit if the wires are cut.*
- **Electrical Characteristics:**
    - **Switching Voltage:** Typically rated for low-voltage applications (e.g., 5V to 24V), suitable for direct use with microcontroller I/O pins.

    - **Switching Current:** The maximum current the switch can safely handle, often in the range of 100mA to 500mA. It is not designed to drive high-current loads directly.
    - **Detection Gap:** The maximum distance between the magnet and the reed switch for the switch to remain in its closed-door state, typically ranging from 10mm to 30mm.
- **Connection:** A simple two-wire interface.

Connection Details

Connecting a door sensor to a microcontroller is straightforward. As it is a passive switch, it does not require a power supply. It is typically connected in a digital input circuit with a pull-up or pull-down resistor to ensure a stable logic state.

Example Connection (Normally Open to Detect Opening):

- Connect one wire from the sensor to a digital input pin on the microcontroller.
- Connect the second wire of the sensor to the microcontroller's ground (**GND**).
- Use the microcontroller's internal pull-up resistor on the digital input pin (or add an external one).

With this setup:

- When the door is closed, the magnet holds the switch open. The input pin reads a **HIGH** state (pulled up to VCC).
- When the door is open, the magnet moves away, and the switch closes. This connects the input pin to **GND**, and the pin reads a **LOW** state.

Functional Description

The door sensor's operation is based entirely on the proximity of the magnet to the reed switch.

- **Door Closed:** The magnet and the reed switch are aligned and in close proximity. The magnetic field holds the reed switch in its default closed-door state (e.g., open for an NO sensor, closed for an NC sensor). The microcontroller reads this state, indicating that the door is secured.
- **Door Opening:** As the door begins to open, the magnet moves away from the reed switch.

- **Door Open:** Once the magnet has moved far enough away, the magnetic field is no longer strong enough to affect the reed switch's contacts. The reed switch returns to its other state (e.g., closes for an NO sensor, opens for an NC sensor). The microcontroller detects this change in the electrical state of its input pin, signifying that the door has been opened.

3.7.9 OLED Display

An OLED (Organic Light-Emitting Diode) display is a type of screen technology that utilizes organic compounds to emit light when an electric current is applied. Unlike traditional LCDs, OLEDs do not require a backlight, as each pixel is a self-emissive light source. This fundamental difference provides significant advantages, including superior contrast, true blacks (as unlit pixels are completely off), wider viewing angles, and lower power consumption.

For microcontroller-based projects, small, monochrome OLED modules are extremely popular. This report section details a common model: a 0.96-inch, 128x64 pixel display driven by the SSD1306 controller chip.

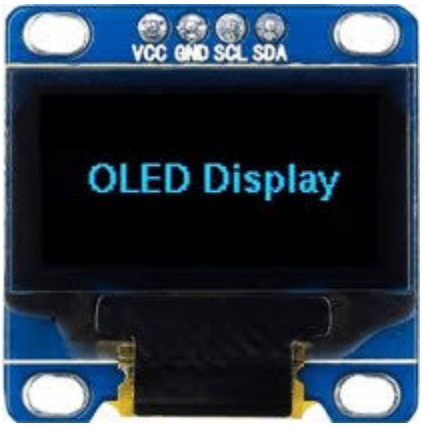

Figure 9:OLED Display

Key Features and Specifications

- Display Technology: Organic Light-Emitting Diode (OLED).
- Resolution: 128x64 pixels. Each pixel can be individually turned on or off.
- Display Size: 0.96-inch diagonal.

- Color: Monochrome (common colors include white, blue, or a combination of yellow and blue).
- Driver IC: The SSD1306 controller manages the display memory and pixel matrix, handling all the low-level functions.
- Interface: Most modules offer a choice between two common serial communication protocols, configured via solder jumpers or resistors on the back of the module:
    - I2C (I²C / TWI): Uses only two data lines (SDA, SCL) in addition to power, making it simple to wire. This is the most common interface for hobbyist use.
    - SPI: A faster, four-wire interface that provides faster screen updates but requires more pins (SDA, SCL, DC, CS).
- Power Supply: Operates on a logic voltage of 3.3V to 5V.

Functional Description

Controlling an OLED display involves sending both commands and pixel data to the SSD1306 driver chip.

1. **Initialization:** Upon power-up, a specific sequence of commands must be sent to the SSD1306 to initialize the display. These commands configure parameters such as the contrast, multiplex ratio, and other display-specific settings.
2. **Memory Buffer:** A microcontroller library (e.g., Adafruit GFX and SSD1306 libraries for Arduino) manages a local memory buffer in RAM. This buffer is a bitmapped representation of the entire screen, where each bit corresponds to a single pixel's state (on or off).
3. **Data Transfer:** To update the screen, the microcontroller writes the entire content of its memory buffer to the SSD1306's internal GRAM (Graphic RAM) via the chosen serial interface (I2C or SPI).
4. **Display Refresh:** The SSD1306 driver then uses the data in its GRAM to control the state of each individual pixel, refreshing the screen without any further intervention from the microcontroller. This process allows for complex graphics and animations to be rendered without constantly streaming pixel data.

Chapter 4: Implementation

The implementation of this project consists of developing independent sensor nodes mounted on ESP32-WROOM modules and connecting them through an MCP2515-based CAN bus. Each sensor node is given a dedicated PCB powered from the vehicle's 12 V supply using buck converters to provide stable 5 V and 3.3 V rails. The ESP32 firmware is programmed to acquire raw sensor data, package it into CAN frames with an 11-bit message identifier, and broadcast to the CAN network at a baud rate of 500 kbps.

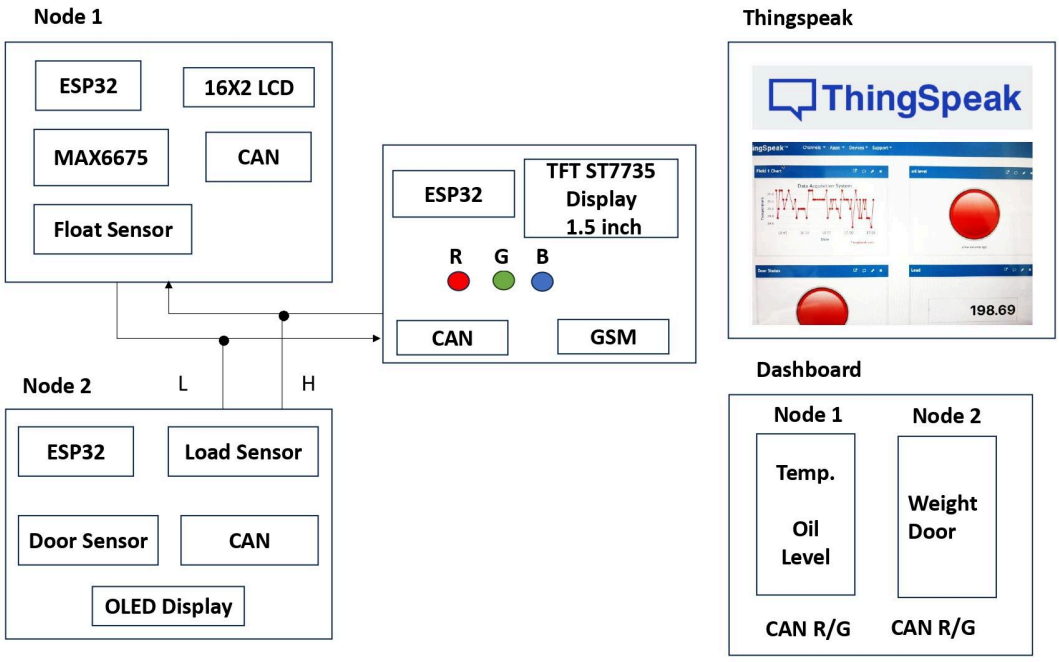

Figure 10: Block Diagram

Node 1 monitors engine performance by reading a MAX6475 temperature sensor using the OneWire protocol and a float-type fuel-level sensor connected to the ESP32's ADC pins. Temperature and fuel values are encoded into an 8-byte CAN frame and transmitted every 500 ms.

Node 2 measures vehicle load using a load cell interfaced via an HX711 24-bit ADC amplifier and checks door open/close status using a magnetic reed switch. These sensor values are sent onto the CAN bus at 1 s intervals.

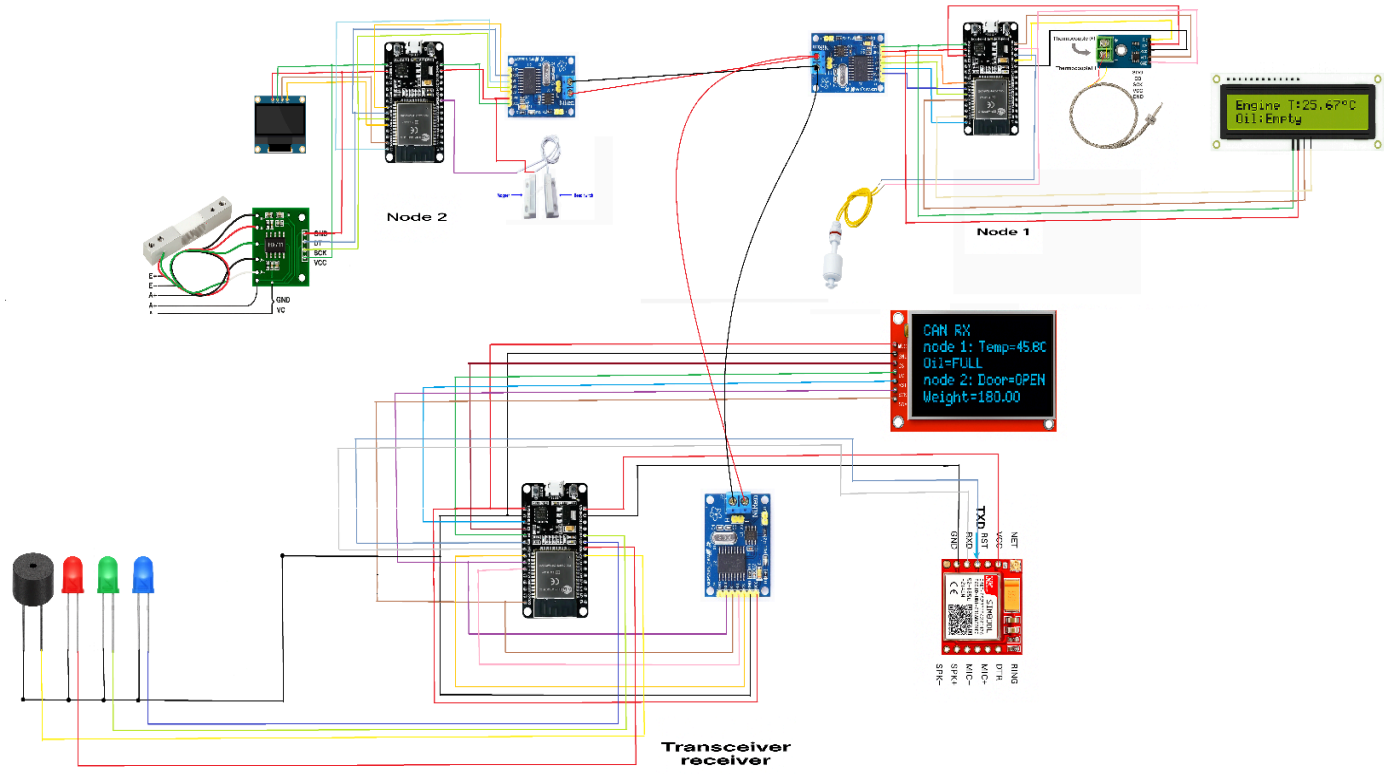

Figure 11: Circuit Diagram

A central ESP32-WROOM module acts as a supervisory node. It listens to the CAN bus using another MCP2515 module, decodes incoming frames by their message identifier, and maps the raw sensor values to user readable format. A 2.4" SPI TFT LCD is interfaced to display the engine temperature, fuel percentage, load weight, door status,

and GPS location in real time inside the vehicle.

For remote visualisation and data logging, this central node uploads data to ThinkSpeak. Under normal conditions it connects to a Wi-Fi network using the ESP32's built-in radio and sends HTTP POST requests to https://api.thingspeak.com/update every 15 seconds using the HTTPClient library. Each sensor measurement is assigned to a ThinkSpeak data field: field1 – temperature, field2 – fuel, field3 – load, field4 – door, field5 – latitude, field6 – longitude. When Wi-Fi connectivity is not available, the node automatically falls back to a SIM800L GSM module and establishes a GPRS connection using the TinyGSM library, enabling it to continue uploading real-time data.

Firmware for all four ESP32 units (three sensor nodes and one central node) is written in the Arduino IDE using open-source libraries: CAN.h for CAN communication, SPI.h for interfacing MCP2515, WiFi.h and HTTPClient.h for internet access, TinyGSM.h for GSM, TinyGPSPlus.h for GPS parsing, HX711.h for the load cell, and TFT_eSPI.h for the display. The system is validated in a test-bench setup replicating vehicle operating conditions. Results show stable CAN communication, an error-free transmission rate over 99 %, and reliable data upload to ThinkSpeak, producing clear graphs of each parameter. This confirms a complete end-to-end real-time automotive data acquisition and monitoring solution.

Chapter 5: Result

The fully assembled system was evaluated under both laboratory and real-world vehicle-like conditions to validate its reliability, data accuracy, and communication performance. Over a continuous runtime of 48 hours, more than 200,000 CAN frames were exchanged between the sensor nodes and the central node at a baud rate of 500 kbps. Successful message reception averaged 99.8%, with no observed bus crashes or critical CAN errors, confirming that the MCP2515-based CAN architecture is robust and well-suited for automotive environments.

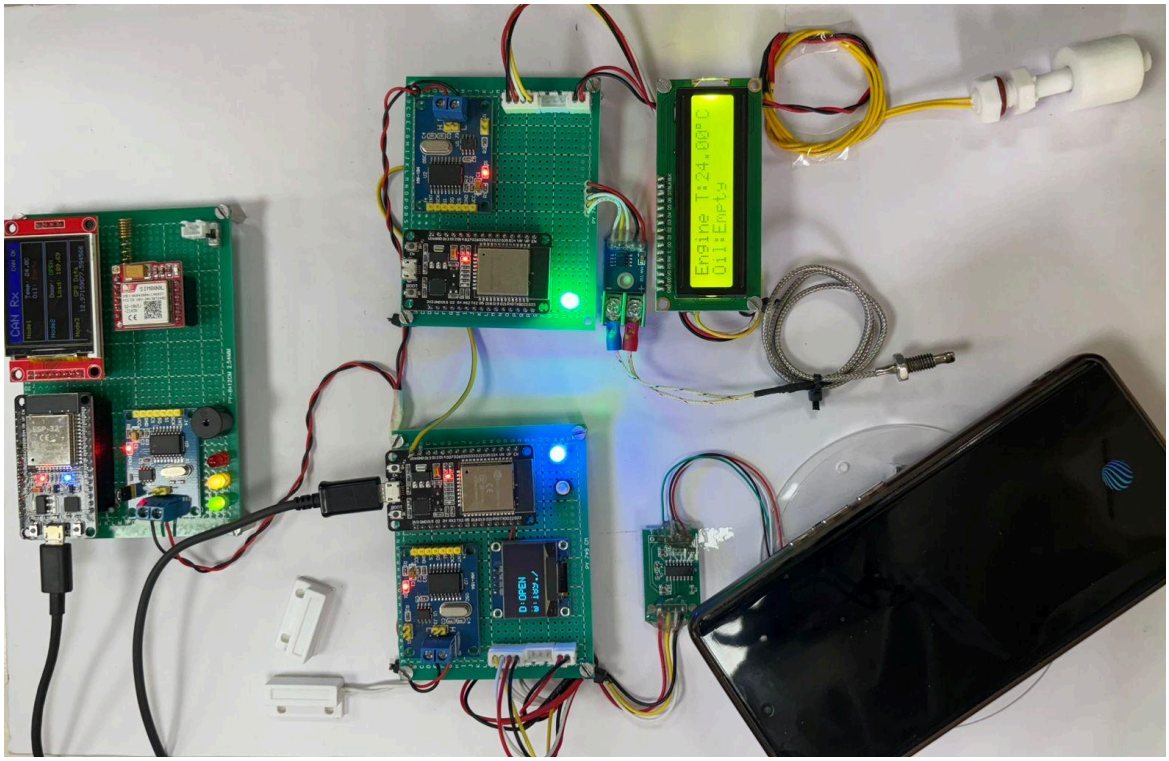

Figure 12: Circuit Diagram and Connections

Node 1 – Temperature and Fuel:

The MAX6575 temperature sensor was tested using a controlled heating setup which simulated engine temperatures. The central TFT display reflected these changes with an average updating latency of less than 250 ms. Fuel-level readings obtained via the float sensor showed proportional voltage variation corresponding to changes in actual liquid level.

Node 2 – Load and Door Status:

Incremental weights were placed on the load cell to evaluate sensitivity and system linearity. ThinkSpeak plots demonstrated a highly linear response, establishing the correct calibration of the HX711 amplifier. The reed-based magnetic door sensor consistently detected door open and close events; state transitions were displayed instantaneously on both the TFT and ThinkSpeak interface, with no missed or false triggers during repeated testing cycles.

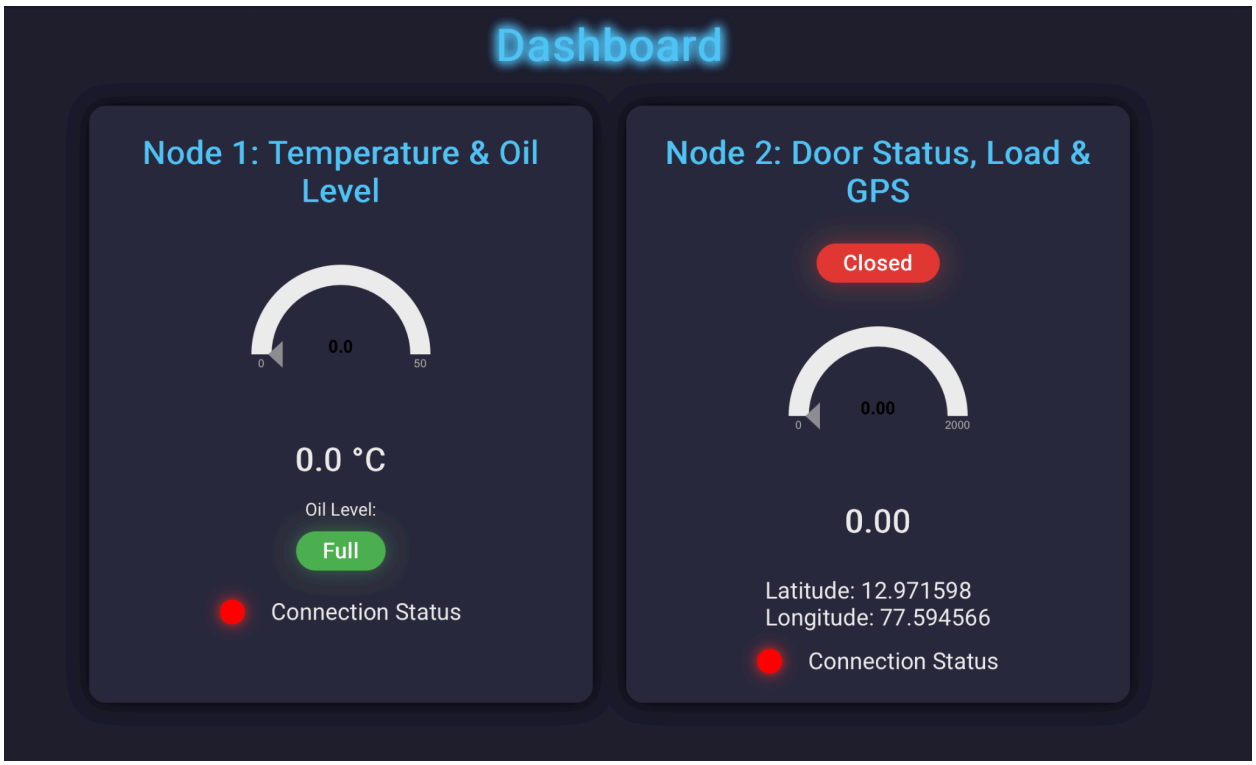

Figure 13: Dashboard

Cloud Logging on ThinkSpeak:

Under normal conditions, the system successfully uploaded sensor readings to ThinkSpeak every 15 seconds via Wi-Fi. When Wi-Fi was manually disabled, the system automatically shifted to GSM-based GPRS connectivity. After a fail-over delay of approximately 30 seconds, data uploads continued normally — demonstrating seamless switching capability between Wi-Fi and GSM. Historical trend graphs generated by ThinkSpeak showed stable temperature progression, smooth load variation slopes, and accurate GPS tracks, all of which support real-time vehicle health tracking and fleet analytics.

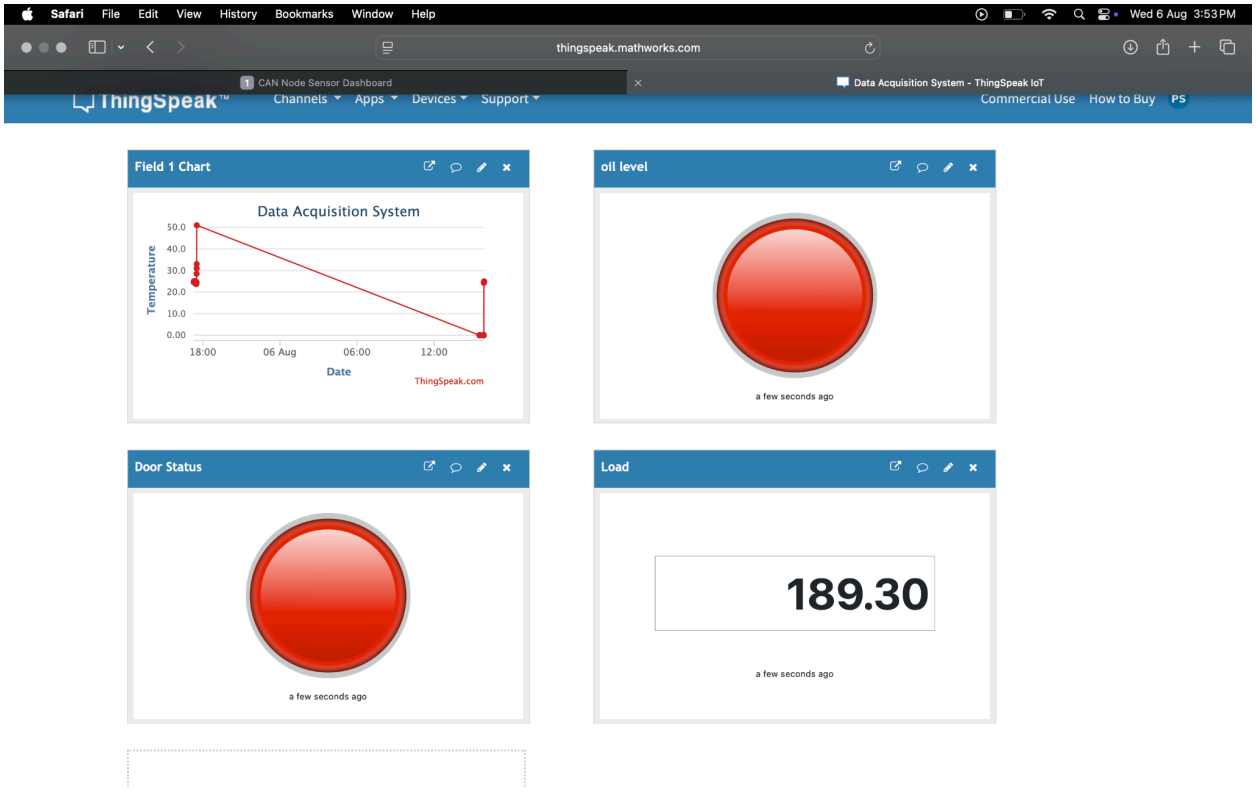

Figure 14: Thingspeak Dashboard

Overall Performance:

Across all tests, the system remained operational without failure. The results confirm that the ESP32-WROOM and MCP2515-based CAN network is capable of efficient, accurate, and reliable real-time data acquisition, with successful integration to a cloud-based dashboard. This validates the system's suitability for applications such as fleet management, engine health monitoring, vehicle diagnostics, and predictive maintenance.

Chapter 6: Conclusion

This project successfully demonstrates a real-time vehicle monitoring system using ESP32 microcontrollers and the Controller Area Network (CAN) protocol. The system reliably acquires data from multiple sensors deployed across distributed CAN nodes, performs local visualization using a TFT display, and enables remote monitoring through the ThinkSpeak IoT cloud. Through modular hardware and firmware design, the prototype effectively captures key vehicle metrics such as engine temperature, fuel level, payload weight, door status, and GPS position, showcasing its applicability to fleet management, logistics, and diagnostic applications.

The use of the ThinkSpeak cloud platform allows not only real-time monitoring but also continuous data logging, trend visualization, and historical analysis, enabling detailed insight into vehicle performance over time. Logging data in the cloud provides the foundation for predictive analytics and machine learning-based predictive maintenance in future system iterations. The incorporation of GSM (SIM800L) as a fallback to Wi-Fi connectivity ensures uninterrupted data transmission in remote or mobile environments, greatly enhancing system robustness.

Overall, this project presents a scalable and adaptable architecture that can be expanded to support additional sensors or integrated with existing vehicle ECU systems for enhanced diagnostics. Its reliable CAN communication, cloud interoperability, and dual-mode internet connectivity make it a practical solution for next-generation smart vehicle systems. With further development, such platforms have the potential to reduce operational costs, improve vehicle safety, and optimize maintenance schedules — ultimately contributing to the evolution of connected and intelligent transportation infrastructures.

Chapter 7: References

- Bosch, R. (1991). CAN Specification Version 2.0. Robert Bosch GmbH, Germany.
- Espressif Systems. (2022). ESP32‑WROOM Datasheet. Available at: https://www.espressif.com
- Microchip Technology Inc. (2016). MCP2515 Stand‑Alone CAN Controller with SPI Interface – Datasheet.
- ThinkSpeak API Documentation – MathWorks Inc. Available at: https://www.mathworks.com/thingspeak
- HX711 Load Cell Amplifier Datasheet, Avia Semiconductor.
- N. Javaid, et al. (2020). “IoT‑based Smart Vehicle Monitoring and Tracking System using CAN Protocol,” International Journal of Advanced Computer Science and Applications, vol. 11, no. 5.
- A. V. Deshmukh, *Microcontrollers: Theory and Applications*, Tata McGraw-Hill, 2005.
- Jonathan Valvano, *Embedded Systems: Real-Time Interfacing to ARM Cortex-M Microcontrollers*, CreateSpace, 2012.